

蚌埠学院

软件系统设计与体系结构综合实训

综合实训设计报告

个人财务管理与可视化看板

项目	请填写
学院	计算机与信息工程学院
姓名	【请填写姓名】
学号	【请填写学号】
班级	【请填写班级】
指导教师	【请填写指导教师】
完成日期	2026 年 9 月

摘要

本项目设计并实现了一套本地优先的个人财务管理与可视化看板“一目”。系统围绕账户、交易、预算、目标和报表形成完整业务闭环。用户可以录入收入、支出和账户间转账，系统根据可追溯交易实时推导账户余额、净资产、月度现金流和预算状态；同时支持账单导入、重复识别、批次撤销、目标进度、原始数据导出和彻底删除。

系统采用 MVC 分层架构和观察者模式。Model 集中封装金额、余额与报表计算规则，Controller 负责交互编排，View 根据只读状态渲染响应式界面。金额统一以整数分存储；本地服务器仅监听回环地址，并通过路径约束、内容安全策略和安全响应头降低风险。项目提供 30 项自动化测试、真实浏览器回归脚本、Java 17 可执行 JAR 和一键启动脚本，满足课程实训的设计、实现、测试、部署与答辩要求。

关键词：个人财务；MVC；观察者模式；可视化看板；本地优先；软件体系结构

1 项目概述

1.1 项目背景

个人财务数据通常分散在银行卡、第三方支付、现金和信用账户中。传统记账工具容易停留在流水堆积层面，用户仍难以快速回答“钱在哪里、钱花到哪里、下一步如何调整”。本项目将交易记录作为唯一可追溯事实来源，用统一账户视图、预算节奏、目标进度和趋势报表把记录转化为可执行信息。

1.2 建设目标

- 统一管理资产账户、负债账户及其动态余额。
- 支持收入、支出、转账的新增、编辑、删除和查询。
- 提供月度预算、财务目标、六个月现金流和支出分类分析。
- 提供 CSV 导入、重复识别、批次撤销、JSON 导出和数据彻底删除。
- 以 MVC 和观察者模式体现体系结构设计，并提供自动化测试与可执行程序。

1.3 范围与边界

课程提交版定位为单用户、本地优先应用，不包含用户注册、云端同步、多人权限、实时金融行情和投资建议。该边界降低敏感财务数据外泄面，也使体系结构、领域规则和测试证据可以在无网络环境下完整演示。

2 需求分析

2.1 功能需求

模块	核心功能	完成
财务概览	净资产、收支、结余率、轨迹、最近交易	是
交易管理	收入/支出/转账、编辑、删除、筛选	是
账户管理	资产/负债账户、期初余额、实时余额	是
预算管理	月度分类预算、阈值状态、删除	是
目标管理	储蓄/还债目标、新增、更新、删除	是
分析报表	六个月现金流、分类结构、本月洞察	是
数据治理	CSV 导入、重复识别、撤销、导出、清空	是

2.2 非功能需求

- 易用：桌面和移动端均可操作；金额可一键隐藏。

- 正确：整数分计算；转账不计入收支；账户引用完整。
- 安全：输出编码；CSV 限制；服务器限制方法、路径和内容类型。
- 可维护：Model、View、Controller 职责分离；计算函数可独立测试。
- 可部署：同时提供 Java 17 JAR、Node 开发服务器和一键运行脚本。

3 总体设计

3.1 MVC 体系结构

表现层由响应式 HTML/CSS 与 DashboardView 组成；控制层由 AppController 统一处理导航、表单、导入导出和高影响确认；领域层由 FinanceStore 与纯计算函数组成；持久化层使用 localStorage，并通过 Java 或 Node 本地服务器交付静态资源。

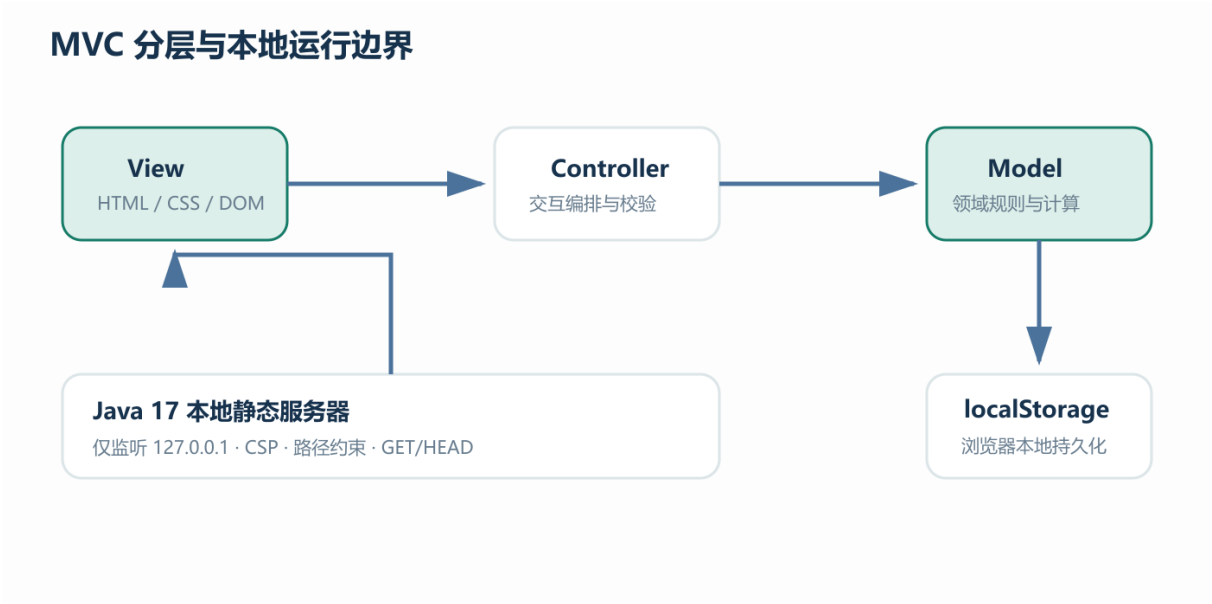


图 3-1 系统 MVC 分层与本地运行边界

3.2 模块职责

层次	主要文件	职责
入口	src/app.js	组装并启动系统
控制层	app-controller.js	交互与业务流程
模型层	finance-model.js	状态、校验、计算和通知
视图层	dashboard-view.js	六个业务页面及可视化
部署层	Main.java	本地静态资源服务与安全头

4 详细设计

4.1 核心数据模型

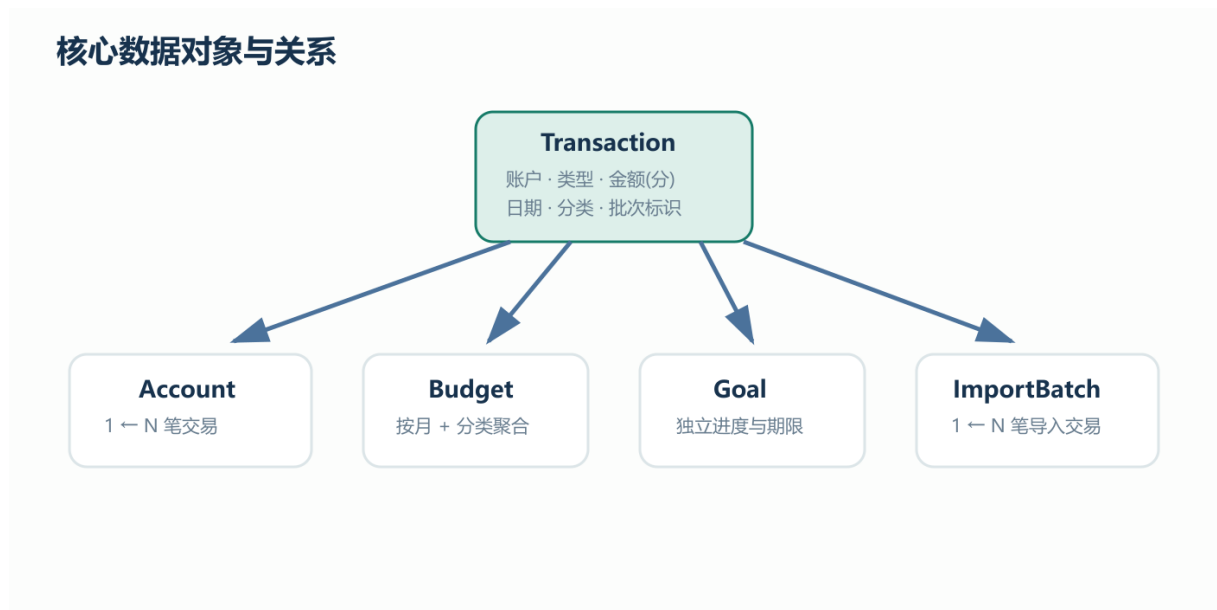


图 4-1 核心数据对象与关联

对象	关键字段	说明
Account	id, kind, openingBalanceCents	当前余额由交易推导
Transaction	type, amountCents, date, accountId	收入、支出、转账统一建模
Budget	month, category, limitCents	月份和分类为逻辑唯一键
Goal	targetCents, currentCents, targetDate	储蓄或还债进度
ImportBatch	id, transactionCount, status	批次可追溯和整批撤销

4.2 关键计算规则

- 资产余额 = 期初余额 + 收入 + 转入 - 支出 - 转出。
- 负债余额 = 期初未偿金额 + 消费支出 - 还款转入。
- 净资产 = 资产余额合计 - 负债余额合计。
- 结余率 = (收入 - 支出) / 收入；收入为零时显示不适用。
- 预算使用率低于 80% 为正常，达到 80% 为提醒，达到 100% 为超支。
- 转账只改变相关账户余额，不进入收入、支出和净现金流统计。

5 设计模式应用

5.1 观察者模式

FinanceStore
作为主题维护观察者集合。所有成功写操作在持久化后发布具名事件，AppController 订阅变更并调用 DashboardView.render。模型无需知道有多少界面组件依赖它，从而降低耦合并保证不同视图同步。

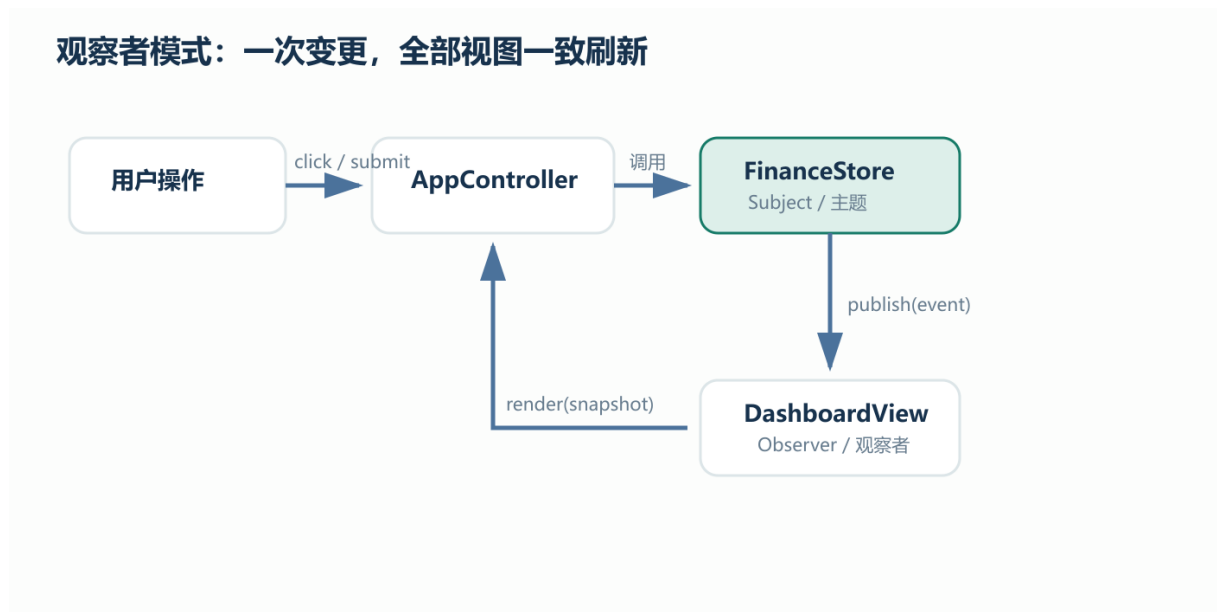


图 5-1 观察者模式协作过程

5.2 策略化计算与适配点

账户余额、月度汇总、预算进度、现金流序列和分类结构均为独立纯函数，可作为可替换的计算策略。持久化通过统一入口与业务操作隔离，当前适配 localStorage，未来可替换为远程仓储。

6 系统实现

6.1 核心界面

系统使用深海军蓝、克制绿色和橙色告警构成统一视觉语言。左侧导航在移动端转换为底部六宫格；概览页将净资产和本月现金流置于首要层级；报表页同时呈现趋势、结构和文字洞察。

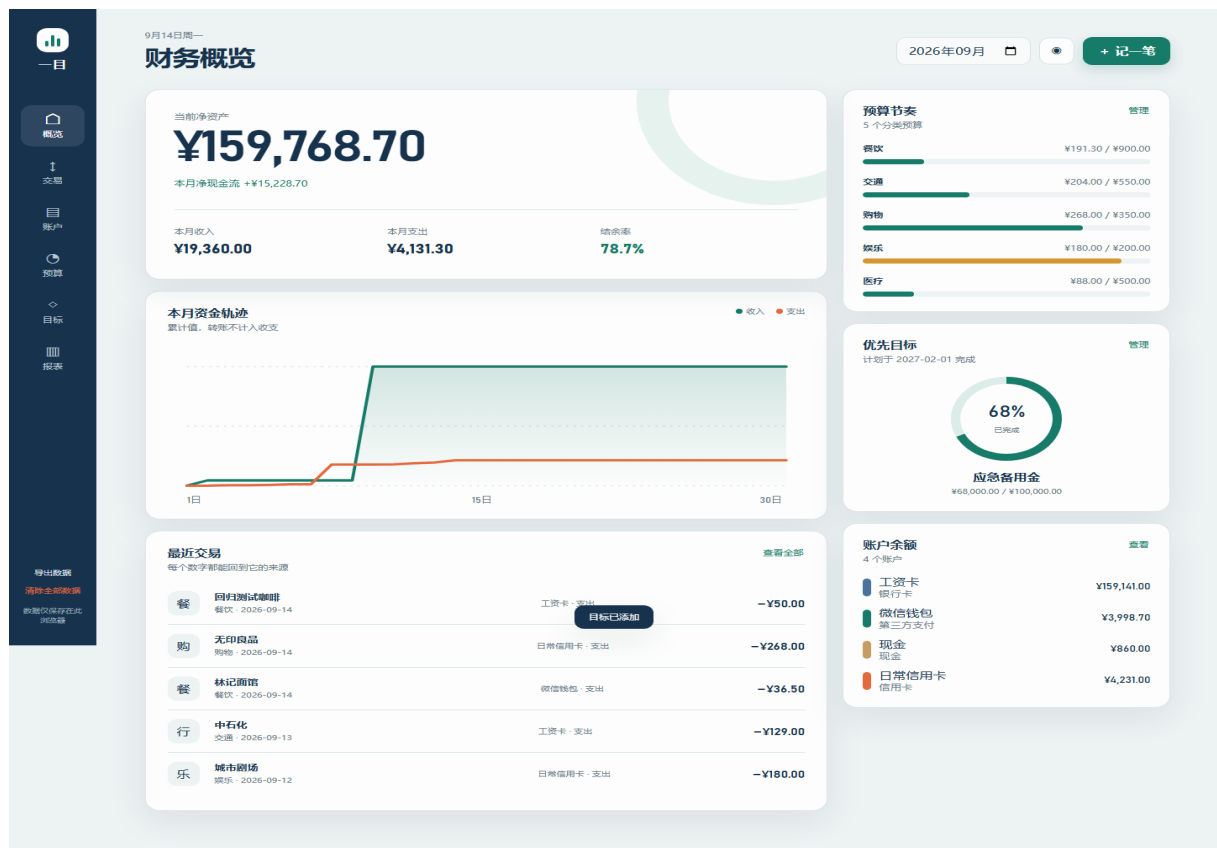


图 6-1 财务概览桌面端界面

6.2 分析报表



图 6-2 分析报表界面

6.3 数据导入与部署

CSV 解析器支持中文表头和带引号字段，将元转换为整数分。重复指纹由日期、金额、账户和商户组合构成。每次导入形成独立批次并写入关联交易，撤销时只删除该批次。Maven 将 Web 资源打入 JAR，Main 类使用 JDK HttpServer 仅绑定 127.0.0.1:4173；run.bat 封装启动命令。

7 测试与质量保证

7.1 测试层次

层次	覆盖内容	结果
领域单元测试	余额、净资产、预算、CSV、观察者、目标、报表、清空	通过
结构与安全	MVC、六页面、表单、路径穿越、隐藏目录、JAR 配置	通过
文档检查	必需章节与内部链接	通过
浏览器回归	编辑、筛选、导入撤销、预算、目标、报表、响应式	通过

7.2 代表性测试用例

编号	场景	预期
T01	资产账户发生收入、支出和转账	余额正确
T02	信用卡消费后还款	负债先增后减
T03	收入为零时计算结余率	显示不适用
T04	重复导入 CSV	提示并跳过
T05	撤销导入批次	只移除关联交易
T06	访问路径穿越地址	服务器拒绝
T07	切换报表页	趋势和分类可见

截至提交版本，Node 自动化测试共 32 项，全部通过；真实 Chromium 回归执行通过，页面无控制台错误。测试命令为 `npm run validate` 和 `npm run test:browser`。

8 安全性与可靠性

- 用户字段在进入 innerHTML 前统一做 HTML 实体编码。
- CSV 上限为 2MB；金额必须为正；账户引用必须存在。
- Java 与 Node 服务器只允许 GET/HEAD，拒绝路径穿越和隐藏目录。
- 响应包含 CSP、nosniff、禁止嵌入和来源策略。
- 清空数据、删除交易和撤销批次都必须确认。
- 产品化时仍需补充认证、授权、服务端事务、加密与备份。

9 项目总结

本项目完成了从需求分析、产品设计、体系结构设计、详细设计到编码、测试和部署的完整实训链路。系统交付可直接运行的Java程序和配套源代码；核心业务闭环、报表分析、数据治理及安全边界均有实现与测试证据。MVC 使关注点清晰分离，观察者模式保证不同视图对同一状态变更保持一致，整数分和纯函数计算提高了财务数据的正确性与可测试性。

后续可以在保持领域接口稳定的基础上增加登录与服务端数据库、可视化字段映射、分类维护、年度预算和多端同步。本次实训说明了体系结构会直接影响代码可读性、变化成本、测试难度和部署可靠性。

10 小组分工

成员	学号	主要工作	贡献比例
【请填写】	【请填写】	需求、架构、实现、测试与文档	100%（个人）
【如为小组请新增】	【请填写】	【请填写实际分工】	【请填写】

参考文献

[1] 软件系统设计与体系结构综合实训指导书，蚌埠学院计算机与信息工程学院。

[2] Gamma E, Helm R, Johnson R, Vlissides J. Design Patterns. Addison-Wesley, 1994.

[3] Fielding R. Architectural Styles and the Design of Network-based Software Architectures, 2000.

[4] WHATWG. HTML Living Standard. 表单、Dialog 与 Web Storage 相关规范。